

Adaptive MoEUT

Marta Muskus Adrian Boguszewski

University of Warsaw

{m.muskus, a.boguszezs2}@student.uw.edu.pl

Abstract

TODO

1 Introduction

Universal Transformers (UTs) are not a new idea (Dehghani et al., 2018). They are a variation of the transformer (Vaswani et al., 2023) architecture that achieves network depth not via stacking multiple transformer blocks, but rather reusing the same blocks in a loop. They are the spiritual ancestor of *Recurrent Neural Networks* (RNNs) in the transformer paradigm and are usually designed either as pure cycles or cycles with additional initial and final layers (see Figure 1).

Proponents of this architecture argue that they generalize better - that the built-in recursion allows the models to decompose structured problems, and generalize to longer sequences (Csordás et al., 2024a). However the weight-sharing also introduces some scaling-related problems, because a UT that loops N times is equivalent in depth to a N layer VT, but only has $\frac{1}{N}$ parameters. One of the ideas to combat this issue is to make the UT’s layer N times wider, which would result in a usually prohibitive jump in compute and memory requirements. Another approach is to modify the UT’s architecture to introduce some sparsity like in *Mixture-of-Experts* (MoE) (Shazeer et al., 2017). This results in a larger parameter count that is arbitrarily scalable, while fixing the compute and memory requirements.

Mixture-of-Experts Universal Transformer (MoEUT) (Csordás et al., 2024a) is a UT architecture that injects sparsity to both the attention computation via *SwitchHead* (Csordás et al., 2024b), and the feedforward layer via σ -MoE.

In this work we attempt to extend the MoEUT design with *Adaptive Compute Time* (ACT) by using the Stick-breaking Halting algorithm proposed

in the *Sparse Universal Transformer* (SUT) (Tan et al., 2023).

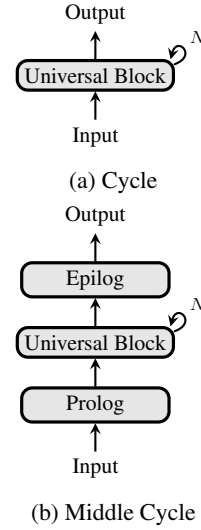


Figure 1: UT Architectures

2 Adaptive MoEUT Architecture

General idea. Our architecture (see Figure 2) incorporates ACT inspired by the SUT. The mechanism maintains two separate token streams throughout the recursive computation: h (the active hidden state) and s (the expected state of the embedding). At each loop iteration, a halting probability is calculated for every active token. Each token accumulates this probability between loops. Once a token’s cumulative probability crosses a predefined threshold it is halted. When halted, a token stops being actively processed through the transformer blocks (it is dropped from the active mask to save compute and is no longer used as a query). However, its expected state s is preserved and continues to be passed as a key and value, allowing the remaining active tokens to attend to it. This ensures that context is maintained across sequence positions without wasting FLOPs on already-resolved tokens.

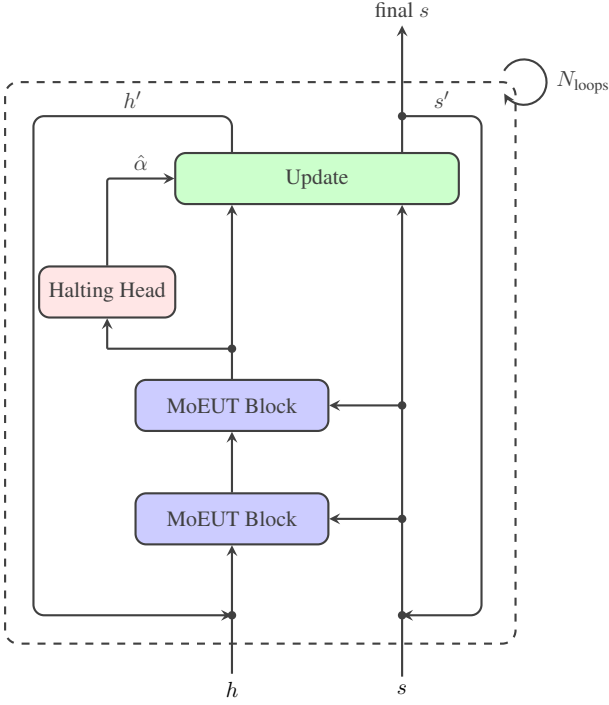


Figure 2: Adaptive MoEUT

Halting Head. The halting probability for each active token is determined by a dedicated Halting Head. This is a lightweight module consisting of a single linear layer followed by a Sigmoid activation, which takes the active hidden state h as input. To encourage the model to leverage the full depth of the recursive blocks early in training, the Halting Head’s weights are initialized to zeros and its bias to a negative value (e.g., -3). This deep-start initialization forces the initial halting probabilities to be small (for a -3 bias it’s $\approx 5\%$), ensuring that the model explores deeper loops before it learns to efficiently halt and optimize its compute time.

Update algorithm. The update mechanism manages the dynamic computation by calculating the incremental halting probabilities (α) from the Halting Head outputs ($\hat{\alpha}$), and subsequently updating the expected states (s) and the active mask. Because the model utilizes soft-halting, the final representation is a probability-weighted combination of the states across the layers a token traversed. To maximize inference efficiency, the algorithm is vectorized and relies on a boolean mask M to compute updates strictly for the active tokens. The process is defined in Algorithm 1. An additional, expected loop depth (per token) penalty is added to the loss, to encourage skipping computations.

Algorithm 1 Adaptive Halting and State Update Mechanism

Require: Initial hidden state h , max loops L , threshold τ

```

1:  $s \leftarrow h$ 
2:  $P \leftarrow \mathbf{0}$  ▷ Accumulated probability
3:  $W \leftarrow \mathbf{0}$  ▷ Weighted previous states
4:  $M \leftarrow \mathbf{1}$  ▷ Boolean active mask
5: for  $l = 1$  to  $L$  do
6:    $h[M] \leftarrow \text{MoEUT\_Block}(h[M], s[M])$ 
7:    $\hat{\alpha}[M] \leftarrow \text{HaltingHead}(h[M])$ 
8:    $\alpha[M] \leftarrow \hat{\alpha}[M] \odot (1 - P[M])$ 
9:    $s[M] \leftarrow (1 - P[M]) \odot h[M] + W[M]$ 
10:   $P[M] \leftarrow P[M] + \alpha[M]$ 
11:   $W[M] \leftarrow W[M] + \alpha[M] \odot h[M]$ 
12:   $M \leftarrow P < \tau$ 
13:  if not any( $M$ ) then
14:    break
15:  end if
16: end for
17: if any( $M$ ) then
18:    $R[M] \leftarrow 1 - P[M]$  ▷ Remainder for tokens
   reaching max depth
19:    $s[M] \leftarrow R[M] \odot h[M] + W[M]$ 
20: end if
21: return  $s$ 
```

KV caching. To support efficient autoregressive generation alongside the adaptive compute mechanism, a specialized caching strategy is used. Instead of maintaining independent caches for each potential loop depth, the cache is shared across all loop iterations. Throughout the dynamic computation, the cache continuously updates to store only the last processed state of every token. Once a token halts, its final computed state persists in this shared cache, allowing active tokens to efficiently attend to it during subsequent generation steps. This strategy attempts to unlock arbitrarily deep recursion with a constant size cache.

3 Experiments

3.1 Baseline Architecture

To establish reference points for the Adaptive MoEUT, we train two parameter-matched baselines: a standard dense Transformer and a standard MoEUT. Both are decoder-only autoregressive language models sharing the same foundational configuration to ensure any quality or efficiency gaps reflect the architecture rather than the capacity. See Table 1.

Dense Transformer. The dense baseline is a standard pre-LN decoder-only Transformer. This model serves as the backbone of the comparison, dictating the parameter count that the MoEUT configuration is subsequently chosen to match. See Table 2.

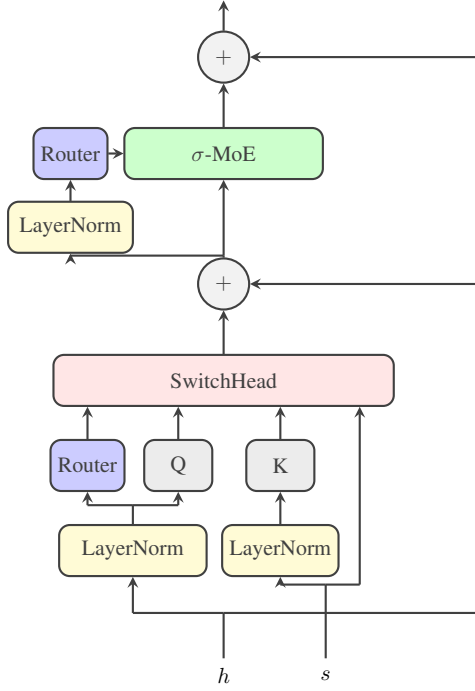


Figure 3: MoEUT Block. The original design was modified to account for the s that are attended to. They have their own LayerNorm and are passed as keys and values to the SwitchHead layer (instead of doing self-attention like in the original MoEUT).

Shared Parameter	Value
Tokenizer	BPE (8,192 tokens)
Model Dimension (d_{model})	512
Context Length	2,048 tokens
Embeddings	Tied input/output
Positional Embeddings	RoPE
Non-embedding Parameters	$\sim 33.6\text{M}$

Table 1: Shared architectural parameters for both baselines.

Dense Parameter	Value
Layers	8
Attention Heads	16
Head Dimension	64
FFN Type	Two-layer ReLU
FFN Dimension (d_{ff})	2,048

Table 2: Dense Transformer configuration.

MoEUT. The MoEUT baseline replaces the dense stack with a Universal Transformer combining extreme weight sharing and sparse activation. It repeatedly applies a group of distinct blocks to achieve the equivalent depth of the dense model with fewer unique parameters. Each block utilizes token-choice sparsity in both its feed-forward network (σ -MoE) and its attention mechanism (SwitchHead). See Table 3.

MoEUT Parameter	Value
Unique Blocks (G)	2
Iterations (R)	4
Total Layer Applications	8
FFN: σ-MoE	
Total Experts (N_E)	112
Expert Width	128
Active Experts per Token (K)	8
Active Hidden Width	1,024 (8×128)
Entropy Regularizer	0.01
Attention: SwitchHead	
Attention Heads	4
Head Dimension	128
Total Value/Output Experts (N_A)	3
Active Experts per Head (K_A)	2
Entropy Regularizer	0.001

Table 3: MoEUT configuration.

3.2 Training Setup

Both models are trained from scratch under an identical regime on a LAMBADA-decontaminated subset of the Colossal Clean Crawled Corpus (C4). Contamination is removed by dropping any document sharing a 13-gram with a LAMBADA passage. The models optimize the cross-entropy language-modeling loss, with the MoEUT model adding its respective entropy regularization terms to the final calculation. All runs are carried out in fp32.

Training Parameter	Value
Optimizer	AdamW
β	(0.9, 0.95)
Weight Decay	0.1
Peak Learning Rate	3×10^{-4}
Learning Rate Schedule	Cosine decay
Warmup Steps	1,000
Gradient Clipping	1.0
Training Steps	100,000
Batch Size	16,384 tokens (32×512)
Total Tokens	$\approx 1.6\text{B}$

Table 4: Shared training setup and hyperparameters.

4 Evaluation

4.1 Baselines

We evaluate both baselines zero-shot on two standard benchmarks. On LAMBADA (Paperno et al., 2016) we report last-word *accuracy* (the model must greedily predict every token of the final word) and target-token *perplexity*. On HellaSwag (Zellers et al., 2019) we report *accuracy* under the length-normalized scoring rule, selecting the candidate ending with the lowest mean per-token cross-entropy. To compare efficiency we also benchmark steady-state inference, reporting per-forward latency and throughput; both models are timed on the same GPU at the same precision (fp32) with discarded warmup iterations and CUDA synchronization, so the wall-clock gap reflects kernel efficiency rather than a difference in nominal FLOPs. Results are summarized in Table 5.

Both baselines perform very similarly in terms of task quality: MoEUT slightly outperforms the dense model on LAMBADA accuracy (18.8 vs. 18.2) though achieves higher perplexity (12.53 vs. 11.92), and both sit close to the 25% random baseline on HellaSwag, as expected at this parameter scale. The main difference is efficiency: despite being parameter- and FLOP-matched, MoEUT is $1.05\times$ slower per forward pass (41.6 vs. 39.6 ms) because its `cvm` expert kernels are less hardware-efficient than the dense model’s matmuls. This inference-time overhead is precisely what our *fully adaptive* model expects to decrease, by spending fewer layer applications on tokens that do not need them.

4.2 Adaptive MoEUT

We experienced a catastrophic decrease in accuracy on the LAMBADA dataset.

Based on the statistics collected during training we can confidently conclude that the model collapsed to a single loop for all tokens.

5 Conclusions

1. We observed that introducing an ACT mechanism into a UT with a naive extra term in the loss caused model collapse. Perhaps additional search for an optimal weight, for the ACT loss would provide better results, however it was not possible in our compute-constrained setting.
2. We observed that theoretical FLOP savings

do not necessarily correlate with wall time reduction or increase in token throughput. We suspect that the main issue was that we introduced tensor slicing into the computations, which resulted in large number of memory operations and irregularly sized tensors not optimal for GPU hardware.

3. We suspect that training the model for longer might result in better scores as the training objective for Adaptive MoEUT was significantly harder. Instead of only optimizing for language modeling it also had to balance the ACT constraint.

6 Further Work

1. Finding an appropriate way to balance the ACT loss term throughout the training. Some of the options are dynamic weighting of the ponder penalty to be lower in the beginning and slowly increasing as the model explores different recursion depths.
2. Testing how larger *maxloops* parameter influences the training.
3. Optimizing the kernels for variable shape tensor computations.

References

- Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. 2024a. *Moeut: Mixture-of-experts universal transformers*. *Preprint*, arXiv:2405.16039.
- Róbert Csordás, Piotr Piękos, Kazuki Irie, and Jürgen Schmidhuber. 2024b. *Switchhead: Accelerating transformers with mixture-of-experts attention*. *Preprint*, arXiv:2312.07987.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. 2018. *Universal transformers*. *CoRR*, abs/1807.03819.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. 2016. *The LAMBADA dataset: Word prediction requiring a broad discourse context*. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1525–1534.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer*. *CoRR*, abs/1701.06538.

Model	Params (non-emb.)	LAMBADA		HellaSwag	Latency	Throughput
		Acc. $\uparrow$	Ppl. $\downarrow$	Acc. $\uparrow$	ms/fwd $\downarrow$	tok/s $\uparrow$
Dense Transformer	33.6M	18.2	11.92	26.7	39.6	103,492
MoEUT	33.7M	18.8	12.53	26.9	41.6	98,472
Fully Adaptive MoEUT (ours)	33.7M	5.6	37.45	25.9	35.7	114,772

Table 5: Zero-shot evaluation and inference efficiency of the parameter-matched models. All models are trained for 100,000 steps on decontaminated C4 and evaluated with a maximum context of 2,048 tokens; latency and throughput are measured in fp32 at batch 8×512 tokens.

Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. 2023. [Sparse universal transformer](#). *Preprint*, arXiv:2310.07096.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. [Attention is all you need](#). *Preprint*, arXiv:1706.03762.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. [HellaSwag: Can a machine really finish your sentence?](#) In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800.